Design layer

the whole layer, as one document

Contents

1 Saga PostgreSQL storage 4

1.1 Foundation 4
1.2 Pending updates 5
1.3 System at a glance 5
1.4 Pool ownership and configuration 6
1.4.1 Connection topology and capacity 7
1.5 Persisted row model 7
1.6 Row state and ownership 9
1.7 Atomic storage operations 10
1.7.1 Conditional writes and refusal precedence 11
1.8 Effect timing and recovery 13
1.9 Time and concurrency 14
1.10 Failure and diagnostic contracts 15
1.11 Schema migration 16
1.11.1 Tables and discovery index 16
1.11.2 Application migration tooling 16
1.12 Release discovery and retention 17
1.13 Verification and extension ports 18

Glossary 19

Saga PostgreSQL storage

1 Saga PostgreSQL storage

1.1 Foundation

Persist Saga executions across processes and nodes through one application-owned PostgreSQL pool.

GOAL

Preserve execution progress across runner loss

Store checkpoints and fenced ownership so a compatible Saga driver can recover the last accepted state.

NO-GOAL

Workflow scheduling and effect reconciliation

Saga owns workflow transitions and recovery evidence. Applications own wakeups, business effects, database operation policy, and delivery coordination.

NO-GOAL

Pool lifecycle and database administration

The adapter borrows a connection handle. The application owns supervision, credentials, connection capacity, database privileges, backups, replication, and failover.

INVARIANT

Only a current claim can replace progress

A checkpoint write matches execution id, claim generation, owner token, expected revision, and observed cancellation in one atomic mutation.

mechanism

INVARIANT

Cancellation preserves checkpoint evidence

Recording cancellation leaves checkpoint bytes, revision, phase, generation, and ownership unchanged.

mechanism

INVARIANT

Claim and lease presence agree

An execution row has both an owner token and lease deadline, or neither.

mechanism

PRINCIPLE

Borrow application infrastructure explicitly

A reusable storage value serves every execution in its schema without creating another pool or imposing a PostgreSQL dependency on core Saga.

PRINCIPLE

Keep uncertainty distinct from absence

Lost replies and interrupted external actions require reconciliation. A lease and a conditional write do not establish exactly-once external effects.

1.2 Pending updates

ruling **Operation deadlines need a topology contract** 2026-10-06

Define which connection topologies the adapter admits and whether the query budget must become one total adapter-operation deadline. Multiple attempts and refusal diagnosis can outlast one query budget; transaction-scoped connections bypass per-query pool deadlines. Saga's watchdog bounds its wait, not remote mutation certainty.

ADR-0004

verify **Distributed and media guarantees need evidence** 2026-10-06

Establish multi-VM contention, node partition, database restart, failover, and acknowledged-write retention under explicit deployment settings. The retained conformance tests use one VM and a disposable database with media durability disabled.

ADR-0003

ruling **Extended persistence authority and retention** 2026-10-06

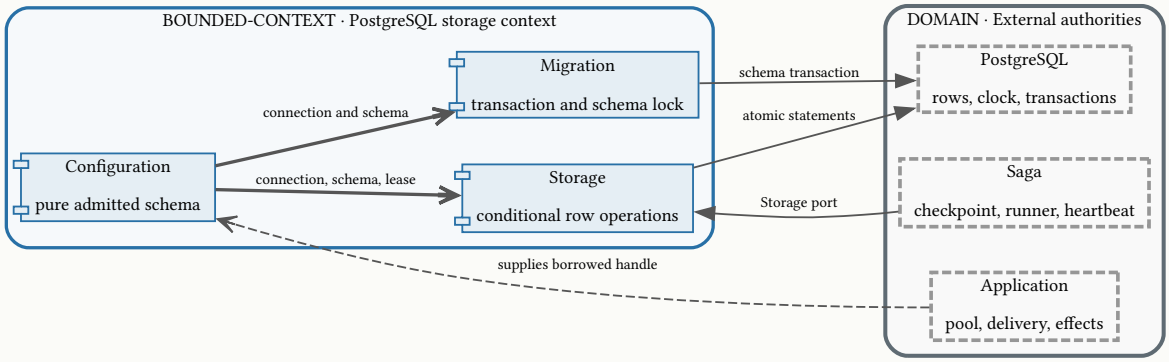
Specify adapter support when Saga admits historical forks, durable approvals, checkpoint conversion, independent children, journal retention, or completed-execution compensation authority. The existing row and storage port supply neither distinct journal retention nor those admission ledgers; their parent contracts remain retained intent.

ADR-0005

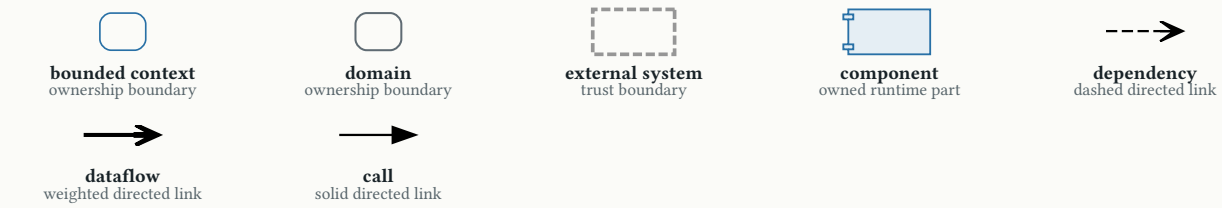
1.3 System at a glance

The PostgreSQL context translates Saga's storage port without owning Saga's workflow meaning.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L2 · CONTEXTS Ownership at the persistence boundary



LEGEND



One local context contains configuration, migration, and storage units. Saga supplies the workflow protocol; PostgreSQL supplies serialization and lease time.

saga_postgres

responsibility

Translate Saga Storage into PostgreSQL execution persistence and fenced claims.

interface

One public module with Config, schema admission, migration, diagnostic rendering, and Storage construction.

interactions

Borrows pog.Connection; uses PostgreSQL statements; declares Saga renewal callbacks.

invariants

No adapter process registry, pool startup, or workflow decoding is required.

failure

Typed schema/migration failures and Saga storage errors preserve caller decisions; unresolved replies preserve uncertainty.

Boundary	Relationship and authority	Contract owner
Adapter ⟶ Saga	Conformist: Saga defines storage values and workflow permission; adapter enforces database mutations.	Saga storage and claim lifetime
Adapter ⟶ PostgreSQL/pog	Anti-corruption layer: PostgreSQL rows and driver failures become Saga Stored, Claim, and Error values.	This layer's row, operation, and failure units.
Application ⟶ adapter	Customer/supplier: application supplies pooled connection, schema access, and lease policy; adapter supplies persistence.	Pool ownership and configuration.

- Saga owns durable execution, checkpoint, claim, and revision semantics. This glossary defines only the PostgreSQL projection and its lease/migration vocabulary.
- The aggregate owner here is the PostgreSQL store for each Execution row. Its database/schema scope separates identical execution ids; an id alone does not locate a database or authenticate a caller.
- The package targets Erlang and depends on Saga, Sinal's transitive identity vocabulary, Gleam libraries, pog, and pgo. It has no Grind, Fabric, Relay, Blueprint, provider, or scheduling dependency; ADR-0001 records the package boundary.

1.4 Pool ownership and configuration

Construction chooses a namespace and lease without opening a connection or starting a process.

Configuration

responsibility

Retain a borrowed connection, admitted schema name, and effective lease.

interface

`config(connection)`, `with_lease(config, Duration)`, `with_schema(config, String)`, and `storage(config)`.

interactions

Produces closures over quoted table names and declares renewal to Saga; migration uses the same config.

invariants

Config is opaque; schema is validated before interpolation; every setting is immutable.

failure

Unsafe names return InvalidSchema(original input); construction checks neither reachability nor installed schema.

Value	Default and refinement	Ownership or limit
Borrowed connection	Required pog.Connection	Pool start/stop/supervision and connection capacity belong to the application.
Schema name	`public`; ASCII lowercase letters, digits, underscore; length 1–63; first character not a digit	Identifiers are double-quoted; values remain query parameters.
Lease	30 seconds; Duration converted to integer milliseconds; values below 100 ms become 100 ms	No upper bound or validation against database delays and heartbeat latency.

Value	Default and refinement	Ownership or limit
Renewal interval	Integer lease milliseconds / 3; at least 1 ms	Saga runs the heartbeat; adapter registers the callback.
Storage call wait	Saga's default 5 seconds; configurable on Storage	Does not increase the adapter query budget.
Pool-backed query attempt	4.5 seconds through pog.timeout	Per attempt; multiple statements and retries require a separate total budget.

BEHAVIOR

Schema admission preserves a usable configuration

When The caller selects a schema name.

Then A valid name produces a new configuration.

Then An invalid name is refused with the original name and leaves the prior configuration available.

level: boundary area: Configuration

BEHAVIOR

Short leases receive the minimum supported period

When The caller supplies a lease below the supported minimum.

Then The configuration retains the minimum lease and declares its corresponding renewal interval.

level: boundary area: Configuration

1.4.1 Connection topology and capacity

- The ordinary contract uses a named pool connection. Each successful pool-backed query checks out one connection and returns it afterwards; refusal diagnosis and retries obtain further loans rather than holding a connection across workflow effects.
- Migration checks out one connection for its whole transaction. Pool sizing must allow that transaction, workflow effect queries, checkpoint writes, and renewal to contend without starving ownership refresh.
- The public connection type also admits transaction-scoped connections. Config performs no topology check; a stored transaction handle has the driver's lifetime and transaction semantics, and is unsuitable as a reusable cross-process pool capability.
- The driver routes pool query timeout through pgo's checkout/loan deadline. A query on an already checked-out connection bypasses that per-query deadline; neither that path nor migration gains a total deadline from the timeout argument alone.
- The adapter adds no queue, backpressure, fair admission, or global concurrency budget. Saga bounds admitted workflow actions per execution; application pool settings bound database capacity.
- Pool-handle inspection tests exclude a configured password from Config and Storage representations. Error text is diagnostic and can include server messages or decoded argument detail; it is not a general redaction boundary.

1.5 Persisted row model

Checkpoint progress, ownership, cancellation, and discovery phase are independent state axes.

Stored discovery phase pending suspended finished

Lease ownership status unowned live expired

Stored cancellation intent clear requested

Execution row

aggregate · stateful · postgres-persistence · owned by PostgreSQL store

The store-owned aggregate retaining one durable execution's checkpoint and write authority.

ATTRIBUTES

- authored **Identity** · Execution identifier within a store
  The caller selects the id; creation stores it unchanged as the unique row key.
- observed **Checkpoint** · Opaque checkpoint bytes and revision
  Saga supplies bytes and expected revision; an accepted write derives the next revision at write time.
- derived **Ownership proof** · Generation and optional owner token
  Claim increments generation and generates a token at write time; release clears the token without resetting generation.
- derived **Ownership** · Lease ownership status row-ownership · row-ownership-lifecycle
  Derived at read time from token/deadline presence and the database clock; expiry permits takeover.
- derived **Lease deadline** · Optional database instant
  Claim, commit, or renewal derives a new deadline from the database clock plus the configured lease.
- observed **Cancellation** · Stored cancellation intent row-cancellation · row-cancellation-lifecycle
  Creation starts clear; a caller request records intent without changing checkpoint progress.
- observed **Discovery phase** · Stored discovery phase row-phase · row-phase-lifecycle
  Saga supplies this projection on commit; the adapter uses it for discovery and never decodes workflow status.
- derived **Creation and update times** · Database instants
  Creation records both; claim, commit, release, and cancellation advance update time. Renewal changes only the lease deadline.

RELATIONSHIPS

- n : 1 Belongs to one PostgreSQL store.
- 1 : 1 Projects one Saga durable execution retained in that store.

Store configuration

value-object · immutable · postgres-persistence · owned by Application through adapter constructors

An immutable choice of borrowed connection, schema, and lease.

ATTRIBUTES

- authored **Connection** · Borrowed database capability
  Application construction selects the database target and pool lifetime.
- authored **Schema** · Schema name
  An admitted namespace separates otherwise identical execution identifiers.
- derived **Lease** · Effective positive duration
  The constructor/setter derives the minimum-constrained millisecond duration at write time.

RELATIONSHIPS

- n : 1 Can refer to one shared application pool.

Applied migration

entity · append-only · postgres-persistence · owned by Migration transaction

An append-only marker of an applied schema version.

ATTRIBUTES

- derived **Version** · Positive migration sequence number
  The migration's final statement records its version in the transaction.
- derived **Installed time** · Database instant
  The database records installation time at write time.

RELATIONSHIPS

- n : 1 Belongs to one schema's migration ledger.

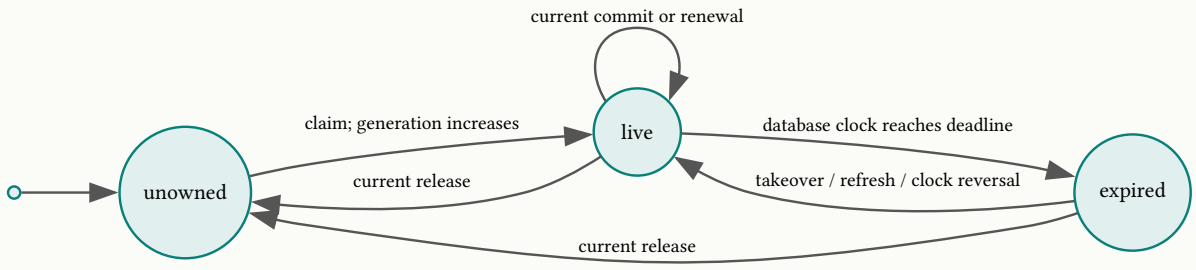
Product or sum	Fields or variants	Refinement owner
Stored projection	Revision × generation × cancelled × bytes	Decoder checks returned types; Saga checks checkpoint meaning. Phase/lease/token/timestamps are not returned by load.
Claim projection	Execution id × generation × owner token	Saga owns its type; this store validates current proof atomically.
Commit input	Expected revision × observed cancellation × phase × bytes	Saga owns the input type; adapter checks agreement and increments revision.
SchemaError	InvalidSchema(original input)	Opaque Config makes unsafe schema interpolation inaccessible through ordinary construction.
MigrateError	MigrationFailed(reason)	Reason is diagnostic; it does not distinguish a lost commit reply.
StorageError	AlreadyExists, NotFound, Busy, StaleOwner, CancellationChanged, Conflict, TimedOut, Corrupt, Unavailable(detail)	The parent storage vocabulary owns variants and callers' interpretation.

- The database schema contains `saga_executions` and `saga_schema_migrations`. An execution row uses text id/token/phase, bigint revision/generation, boolean cancellation, bytea data, nullable timestamptz lease, and nonnullable creation/update timestamps.
- The primary key enforces id uniqueness; check constraints enforce nonnegative revision/generation, the three phase labels, and paired nullability of token/deadline. There is no adapter checkpoint byte bound, id-length bound, or generation-wrap policy.
- Commands are create, claim, commit, release, renew, and cancel; queries are load and unfinished discovery. Migration is a schema command; the adapter publishes no domain event or outbox, and owns no application business entity.

1.6 Row state and ownership

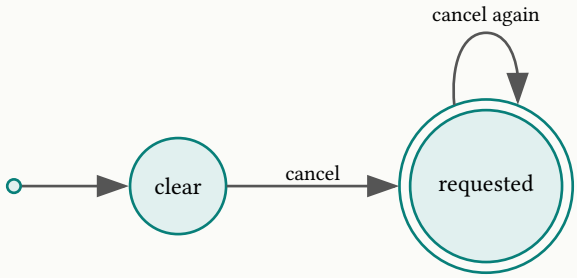
Expiry changes takeover eligibility; replacement changes which claim can write.

STATE MACHINE Unreplaced expired claims retain write authority

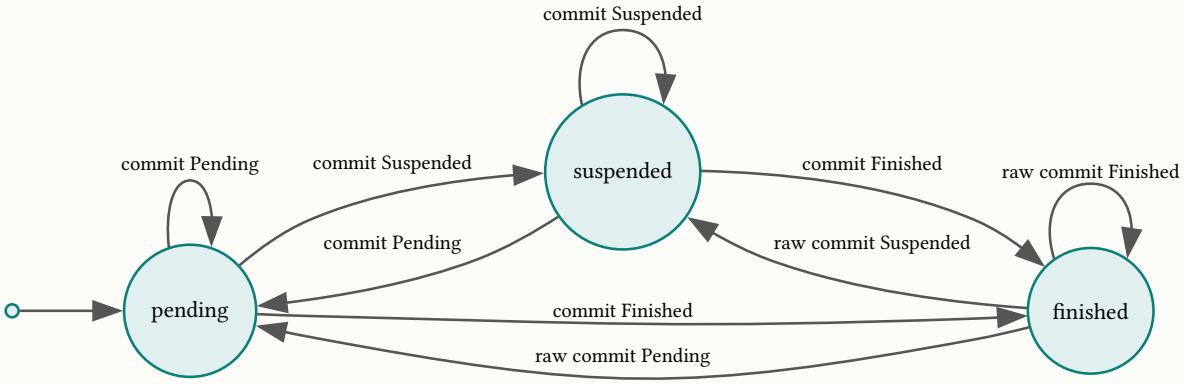


Refresh preserves claim identity; takeover replaces generation and token. Clock reversal can also change derived eligibility and is a deployment timing constraint.

STATE MACHINE Cancellation intent only turns on



STATE MACHINE The adapter accepts the supplied discovery phase



This is the storage-level accepted state set. Saga's compatible checkpoint/driver protocol supplies terminal workflow semantics; the adapter imposes no finished-phase claim or write guard.

- Creation yields revision zero, generation zero, clear cancellation, Pending phase, supplied bytes, and no claim. Only accepted checkpoint commits increase revision; only accepted claims increase generation.
- Ownership, phase, and cancellation are independent axes. Claim does not inspect phase or cancellation; cancellation does not release ownership; Finished is omitted from discovery regardless of ownership.
- A claim is transferable across processes. The adapter neither checks the claiming PID nor monitors it; generation and token equality establish current write authority.
- Lease expiry alone does not refuse commit, renewal, or release. Those operations match current generation/token; a successor claim revokes all three rights of the prior claim.
- When renewal fails or is delayed, a live runner can become eligible for takeover. Database clock correctness, pool capacity, scheduling delay, and successful refresh are conditions of exclusivity over time; ADR-0003 records the lease trade-off.

1.7 Atomic storage operations

One accepted mutation changes one execution row; diagnostics can require another statement.

Storage operations

responsibility

Apply the parent storage protocol to rows without workflow decoding.

interface

Storage closures for create/load/claim/commit/release/cancel/unfinished plus renewal.

interactions

Parameterized statements borrow pool connections; decoders return Saga Stored/Claim values.

invariants

Accepted mutations are atomic; condition refusal leaves progress unchanged.

failure

No-row results produce typed protocol refusals; driver errors remain distinct from refusal.

Operation	Atomic predicate and mutation	Result
create	Insert id with initial state; on duplicate do nothing.	Stored or AlreadyExists; existing bytes remain intact.
load	Read revision, generation, cancellation, and bytes by id.	Stored or NotFound; malformed returned types are Corrupt.
claim	Match id and absent token or expired deadline; increment generation; generate random UUID token; refresh deadline/update time.	Claim and latest Stored; otherwise diagnose NotFound or Busy.
commit	Match id, generation, token, revision, and cancellation; increment revision; replace bytes/phase; refresh deadline/update time.	Stored; otherwise diagnose NotFound or ordered refusal.
release	Match id, generation, and token; clear token/deadline; update time.	Nil; otherwise NotFound or StaleOwner.
renew	Match id, generation, and token; refresh deadline only.	Nil; otherwise NotFound or StaleOwner.
cancel	Match id; set cancellation true; update time.	Nil or NotFound; repeated requests succeed.
unfinished	Nonfinished phase and absent or expired claim; oldest creation then id; positive row limit.	List of candidate ids; nonpositive limit returns an empty list without a query.

1.7.1 Conditional writes and refusal precedence

Commit predicate

responsibility

Separate changed ownership, changed cancellation, and changed progress.

interface

Parent Claim and Commit; returned next Stored or storage.Error.

interactions

One conditional update followed, on no row, by an owner-state read.

invariants

Generation/token comparison precedes cancellation interpretation; cancellation precedes revision conflict.

failure

A diagnostic read can fail independently or observe a later concurrent state; it is not the failed mutation's original snapshot.

- No-row diagnosis reads generation, cancellation, and token. An absent row returns NotFound; a mismatched or cleared token/generation returns StaleOwner; changed cancellation returns CancellationChanged; otherwise the result is Conflict.
- Claim refusal similarly reads for existence, then returns Busy without reserving anything. Release and renewal refusal read for existence and otherwise return StaleOwner.
- Diagnosis is not part of the mutation transaction. Generation and revision increase, cancellation only turns on, and release clears the token; later competing operations can change the reason observed by the diagnostic read.
- A current claim plus stale revision cannot overwrite progress. After CancellationChanged the driver must reload and observe intent before another checkpoint write; it must not blindly repeat the old observed flag.

BEHAVIOR

Progress replacement requires all current authority

Given The caller holds a claim and an expected checkpoint revision with an observed cancellation value.

When The caller attempts to replace progress.

Then The replacement succeeds only when ownership, revision, and cancellation observation all agree.

Then A refusal preserves checkpoint bytes and revision.

level: boundary area: Conditional progress

BEHAVIOR

Refusal identifies the highest-priority disagreement

Given A progress replacement changes no execution and the current state can be read.

When The adapter diagnoses the refusal.

Then It reports missing execution before evaluating authority disagreements.

Then For an existing execution, changed ownership precedes changed cancellation, which precedes progress conflict.

level: boundary area: Conditional progress

BEHAVIOR

Duplicate creation preserves the first checkpoint

Given An execution identifier already exists in the store.

When The caller creates an execution under that identifier.

Then Creation is refused and the first checkpoint remains available.

level: boundary area: Admission and reads

BEHAVIOR

Current claims exclude competitors until expiry

Given An execution has a claim whose lease deadline lies ahead of the database clock.

When Another caller attempts to claim the execution.

Then The attempt returns Busy and preserves the current claim.

level: boundary area: Ownership

BEHAVIOR

Takeover revokes the earlier ownership proof

Given A current claim has expired.

When A competing caller successfully claims the execution.

Then The generation advances and a new owner token is returned.

Then The earlier claim can no longer commit, renew, or release.

level: boundary area: Ownership

BEHAVIOR

Cancellation records intent without replacing progress

Given The execution exists.

When The caller records cancellation.

Then The cancellation flag is true while checkpoint bytes and revision remain unchanged.

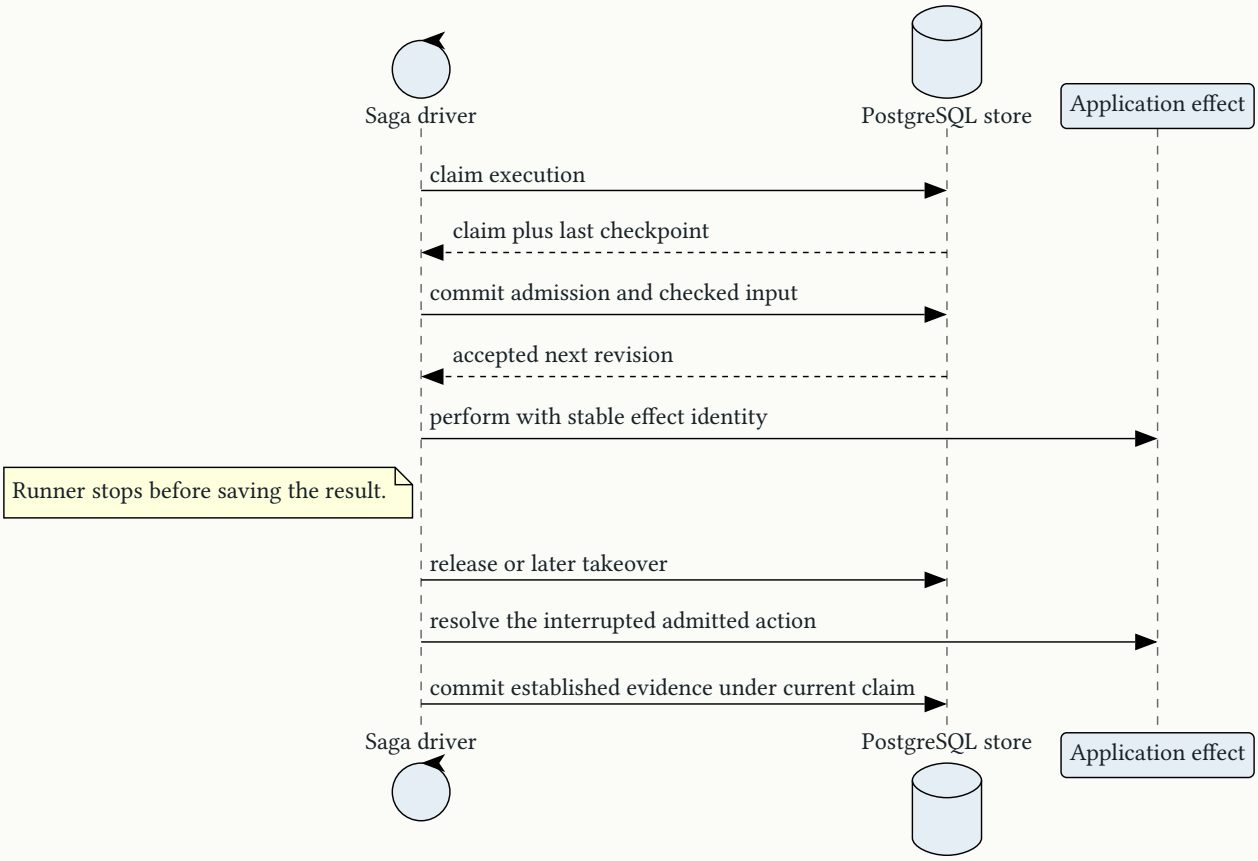
Then Repeated requests succeed without changing ownership.

level: boundary area: Cancellation

1.8 Effect timing and recovery

The accepted checkpoint is recovery authority; a database acknowledgement cannot settle an external effect.

SEQUENCE One interrupted effect across ownership replacement



Saga controls admission and resolver evidence. The adapter controls accepted writes and ownership; it neither invokes effects nor automatically replays them.

Boundary	What becomes authoritative	Uncertainty or later work
Creation reply	Initial checkpoint and id reservation	Creation with a lost reply is reconnected by stable id through Saga's input/definition checks.
Claim reply	New generation/token plus stored bytes	A lost reply can leave an unobserved claim until expiry.
Commit reply	Next checkpoint revision and lease refresh	An unavailable/timed-out reply does not itself establish whether the write took effect.
Effect completion	Application effect evidence	A separate checkpoint records it; the gap requires a resolver after interruption.
Cancel reply	Stored cancellation intent	Saga observes it at persistence boundaries; it does not stop or reverse a remote effect by itself.

Boundary	What becomes authoritative	Uncertainty or later work
Release reply	Vacant ownership	A lost/failed release leaves lease expiry as the recovery fallback.
Migration transaction commit	Schema steps and version markers together	A lost commit reply can leave the migration applied; a new migrate call is safe.

- Conditional write admission is the adapter's effect boundary. Saga's checkpoint and effect permission and recovery rules own checkpoint format, codecs, compatibility stamps, and resolver outcomes.
- An application step may query the same pool, but its business write and Saga's subsequent checkpoint are independent transactions. This adapter exposes no combined business/checkpoint transaction or scheduling outbox.
- Ownership fencing guards later saved writes. An already sent remote request can continue after runner loss or takeover; applications use stable effect identities, downstream deduplication, and resolver evidence.
- The adapter creates no per-checkpoint observation event. Applications consume typed storage/durable results and the parent observation contract rather than treating lack of an event as proof of absence.

1.9 Time and concurrency

Lease time, query attempt time, storage wait, and workflow time belong to different owners.

Query execution

responsibility

Bound pool-backed query attempts and retry only known transactional refusal classes.

interface

Internal run helper with the fixed query timeout and three attempts.

interactions

pog delegates pool checkout/loan timing to pgo; Saga uses a separate call watchdog.

invariants

Parameters carry values; only serialization failure and deadlock SQLSTATEs are automatically retried.

failure

Final query errors map to storage errors; helper/runner timeout can leave remote result certainty unresolved.

Clock or bound	Use	Constraint
PostgreSQL clock_timestamp	Lease deadline, takeover eligibility, creation/update/installation instants	Node clocks do not decide lease eligibility; database clock shifts affect elapsed lease behavior.
Saga runner heartbeat	Refresh every third of effective lease	Scheduling, database availability, and pool capacity can prevent timely refresh.
Query loan deadline	4.5 seconds for each pool-backed attempt	A refusal may issue a second read; each SQLSTATE retry receives a fresh attempt budget.
Saga call watchdog	Default 5-second wait for an operation	Caller may shorten/extend it; it bounds waiting and may stop the helper before adapter work completes.
Migration time	One transaction, with selected query timeout arguments of 60 seconds	Checkout, transaction commands, and already checked-out query execution do not form one total budget.
Checkpoint deadline/backoff	Saved Saga workflow instants	Owned by Saga/application; independent from database lease time.
Drive execution budget	Caller-supplied Duration	Parent driver drain/release can follow expiry; reserve cleanup margin separately.

- Store statements target READ COMMITTED. For SQLSTATE 40001 (serialization failure) or 40P01 (deadlock), run retries the same statement at most twice after the initial attempt, without backoff. Other failures and protocol refusals are not automatically retried.
- An accepted row mutation uses the database's statement serialization. A later diagnostic read is separate; discovery neither locks nor claims rows. Multiple drivers or sweepers must still handle Busy and conditional refusal.
- One query attempt budget is not a total-operation deadline. In the retry path three attempts can consume three budgets; a refusal read adds another independently budgeted query sequence.
- Expiry is judged at query execution, with takeover allowed when deadline is at or before the database clock. Refresh uses a new database instant; a slow callback does not renew by itself.
- Saga stops its heartbeat with the runner and reacts to StaleOwner. Transient renewal failures retry at a later interval under Saga policy; the adapter cannot promise that a live but partitioned runner retains ownership.
- ADR-0004 distinguishes the implemented query mechanism from a stronger end-to-end deadline. No total-return, failover consistency, or simultaneous-effect exclusion follows from the shorter default query setting.

1.10 Failure and diagnostic contracts

Callers act on typed classes; detail strings describe driver evidence.

Evidence	Typed result	Caller meaning
Duplicate row creation	AlreadyExists	Reconnect/check the existing execution through Saga.
Unknown execution	NotFound	No row observed by the relevant read or mutation.
Excluded live owner	Busy	Delay or retry under the lease/delivery policy.
Claim proof no longer current	StaleOwner	Stop this owner's progress; reacquire only through normal claim admission.
Cancellation differs from observation	CancellationChanged	Reload before further checkpoint interpretation.
Progress condition disagrees	Conflict	Reload current progress; do not overwrite.
pog QueryTimeout	TimedOut	Waiting failed; a mutating operation can require reconciliation.
Unexpected returned types	Corrupt	The returned projection could not be decoded; checkpoint content validation remains Saga's job.
Other driver failure	Unavailable(detail)	Diagnostic availability/configuration/database failure; no detail parsing for control flow.
Migration transaction failure	MigrationFailed(reason)	No partial committed migration steps; the transaction's commit reply may be ambiguous.

Error translation

responsibility

Preserve storage refusal distinctions and convert driver failure into stable operational categories.

interface

SchemaError, MigrateError, describe_migrate_error; Saga storage.Error from callbacks.

interactions

Formats pg SQLSTATE/name/message, constraints, argument mismatch, and decoder detail.

invariants

Unavailable detail is diagnostic; query timeout and result corruption do not collapse into protocol Conflict.

failure

The adapter has no mutation-receipt or outcome-unknown error variant; callers must not infer absence from a lost reply.

- Serialization/deadlock retries rely on PostgreSQL refusal of the statement's transaction. Within an application-owned outer transaction, transaction abort/lifetime rules also apply; the adapter is not a savepoint manager.
- A checkpoint can decode correctly at the row boundary and still be malformed, incompatible, oversized, or semantically inconsistent to Saga. Parent checkpoint admission owns those refusals and prevents effect permission.

- Schema names and lease are local configuration. Database privileges, missing tables, connection loss, and unsupported database behavior appear at migration/storage execution rather than storage construction.
- Driver exceptions and connection-process exits are not all converted by this adapter's Result mapping. Saga's helper/runner ownership supplies its own lost-process/watchdog outcomes; direct callback users inherit driver behavior.

1.11 Schema migration

Schema changes and their version markers commit together under one per-schema transaction lock.

Migration

responsibility

Create the chosen schema if absent and apply unapplied forward schema steps.

interface

`migrate(Config) -> Result(Nil, MigrateError)`; packaged up/down SQL for external migration tools.

interactions

Borrows one connection for a transaction, sets READ COMMITTED, takes advisory lock, sets local search_path, and executes ordered statements.

invariants

A step's marker shares its transaction with schema changes; concurrent package migration callers serialize per schema.

failure

Roll back on callback failure; lost commit reply may leave the schema applied; a repeat is safe under the marker contract.

Ordered step	Authority or action	Failure boundary
Begin	Acquire transaction connection; set transaction isolation READ COMMITTED	Driver checkout/transaction failures are MigrationFailed.
Lock	pg_advisory_xact_lock(hashtextextended(lock prefix + schema, 0))	Wait belongs to transaction/driver timing; a hash collision may serialize unrelated schemas.
Namespace	Check pg_namespace; create only if absent; set local search_path to quoted target schema	Precreated schema avoids needing database CREATE privilege.
Watermark	Find saga_schema_migrations; read max(version), or zero if absent	Assumes an append-only contiguous migration history rather than verifying it.
Apply	Run ascending packaged versions greater than watermark; first statement takes the same lock; last records version	No automatic migration retry loop.
Commit	Publish tables/indexes/markers together; release transaction-scoped lock	A commit-reply failure is ambiguous; rerun migration.

1.11.1 Tables and discovery index

- Migration version one creates the version ledger, execution table, and partial index on creation time then id for rows whose phase is not Finished. It records version one last.
- The index supports oldest-first unfinished discovery. Lease time is checked dynamically rather than indexed as a fixed expiry bucket; a large population of live unfinished rows can still require filtering.
- The watermark is maximum version, not a list of gaps or migration checksums. A newer recorded version causes this package to apply nothing older and does not establish that an older adapter understands every newer schema.
- Append numbered migrations; preserve released migration statements. Internal statement lists and checked-in SQL up sections must agree statement for statement; ADR-0002 records that dual entry point.

1.11.2 Application migration tooling

- The SQL file uses migration:up/down/end sections. An external tool creates/selects the target schema and runs the up statements in one READ COMMITTED transaction with search_path identifying that schema.
- The up statements take the same lock prefix and current_schema-derived key as package migration, then record the same version. Coordinated tools must select unapplied versions after acquiring the lock; the raw SQL file itself is not repeat-idempotent.

- Package migrate is forward-only. The shipped down section destructively drops the execution and migration tables; rollback execution and data retention are external administrative decisions.
- Schema installation is separate from checkpoint/definition migration. The adapter never transforms stored Saga bytes or checks a workflow compatibility stamp.

BEHAVIOR

Package migration serializes concurrent installation

Given The target namespace follows the packaged migration ledger contract.

When Concurrent callers request migration.

Then Installation is serialized for that namespace and already applied steps are skipped.

Then Failed installation leaves no partially committed step, subject to uncertainty in the final acknowledgement.

level: boundary area: Migration

1.12 Release discovery and retention

Driver lifetime and saved execution lifetime are independent.

Release and discovery

responsibility

Make eligible unfinished executions available to an application-owned next driver.

interface

Current-claim release, renewal callback, and unfinished(limit).

interactions

Saga normally releases on runner completion/stop; an application scans and reconnects candidates.

invariants

Release clears only current ownership; discovery excludes Finished and live claims.

failure

A lost release falls back to lease expiry; candidate scans can race another claim; NotFound and StaleOwner remain distinct.

BEHAVIOR

Release removes only current ownership

Given The caller holds the current claim.

When The caller releases it.

Then Ownership becomes vacant while checkpoint progress and generation remain retained.

level: boundary area: Ownership

BEHAVIOR

Discovery reports candidates without reservation

When The caller requests unfinished executions with a positive limit.

Then The result contains at most that many nonfinished executions without a live claim, oldest first with identifier ordering for equal creation times.

Then The caller must claim a candidate before driving it.

level: boundary area: Discovery

- A killed runner whose drive caller survives can be released promptly by Saga. Complete node loss or release failure leaves the saved ownership until its last lease deadline; the adapter has no process monitor or background sweeper.

- The owner-loss window is one configured lease from the last accepted refresh under a progressing database clock. It is not a promise that discovery or a successor starts within that window.
- One discovery call is a snapshot, not a reservation or a continuous watch. Repeated sweepers can receive the same ids; there is no notification, cursor, automatic wakeup, or fairness guarantee.
- Retention is indefinite here: no TTL, pruner, delete, or archival API removes executions. Finished rows remain loadable and reserve their ids; release does not delete bytes or journal evidence.
- Deleting data with administrative SQL discards recovery and idempotent admission evidence. Applications must decide reference validity, child/compensation authority, backups, and identifier reuse before introducing retention deletion.
- The parent composition contracts retain independent result/journal retention and owner-backed compensation authority. Those contracts do not exist in this schema and are the extension boundary recorded by ADR-0005.

1.13 Verification and extension ports

Protocol conformance and deployment durability require different evidence.

Verification harness

responsibility

Exercise storage semantics against real PostgreSQL without reaching a configured external database.

interface

Parent Nix shell; scripts/test-postgres.sh; public Saga conformance fixture plus adapter tests.

interactions

Creates a temporary PostgreSQL 16 cluster, per-test pools/schemas, and test-only FFI cleanup.

invariants

Plain tests fail without the script-provided database URL; test cleanup stops owned pools and removes the cluster.

failure

Startup/test failure is reported; server log tail preserves database failure evidence before deletion.

Evidence surface	Assertions	Limit
Saga storage conformance	Atomic creation, transferable claims, cancellation races, stale/refused writes, release, live/lost ownership, discovery, killed-runner recovery	One VM; adapter owner-loss allowance is separate from scenario operation timeout.
Configuration tests	Lease/renewal defaults and minimum; default call timeout; pool-handle password exclusion	No general credential/redaction or hostile-server proof.
Migration tests	Concurrent/idempotent installation, independent schemas, invalid names, newer watermark, unreachable target, SQL equality/external application	Current version one; no historical upgrade chain or failover test.
Store tests	Refusal precedence, missing execution, phase discovery, current renewal, stale proof, blocked query timeout, expired unreplaced claim	Timeout test covers a row lock on the normal pooled path.
Durable tests	Saved result reuse, long live runner heartbeat, prompt killed-runner release, lost-release fallback, resolver recovery, pre-drive cancellation	Lost release is simulated; no multi-node partition or database power loss.
Shared-pool consumer	Five concurrent durable runs plus application writes using two connections	Demonstrates borrowed pool composition; does not establish performance/fairness under overload.

- The shell script clears PG-prefixed settings and the prior test URL before creating its own loopback cluster. It selects a free port, uses trust authentication, disables fsync and synchronous_commit, and traps exit for immediate stop/removal.
- Test support creates unique schemas and pools. Test FFI wraps callbacks in an after clause, stops the owned pool, and escalates to kill after the shutdown wait; FFI is not part of production storage.

- Those settings make tests disposable rather than media-durability evidence. Production acknowledged-write survival depends on the application's PostgreSQL settings, storage, backups, and replication/failover contract.
- The callable boundary inventory is six public adapter functions, seven Storage callbacks, and one renewal callback. Behavior areas are configuration, migration, admission/reads, ownership, conditional progress, cancellation, and discovery; asynchronous UI/delivery, domain-event publication, and provider policy are inapplicable because the adapter owns none.
- A new storage wrapper must preserve renewal and Saga's call-timeout ownership. A new schema step must preserve atomic authority, refusal precedence, opaque bytes, and the packaged SQL equivalence test.
- The parent storage verification owns the common conformance contract. ADR-0003 states the evidence limit; ADR-0005 preserves extension requirements.

Glossary

Every term the layer declares, with the context that owns it.

Borrowed pool

owned by root

An application-owned database connection pool referenced by the adapter without transferring startup, supervision, or shutdown ownership.

Conditional write

owned by root

A checkpoint replacement admitted only when current claim identity, expected revision, and observed cancellation all match.

Discovery candidate

owned by root

An unfinished execution identifier observed without a live claim at the instant of discovery. Discovery reserves no ownership.

Execution row

owned by root

The retained record of one Saga durable execution in a PostgreSQL store. Its identity is the execution identifier within that store.

Lease

owned by root

The period during which a current ownership claim excludes takeover, measured from its last accepted refresh against the database clock. Expiry permits replacement without immediately invalidating the unreplaced claim.

Lease deadline

owned by root

The database instant after which a current claim becomes eligible for takeover.

Migration lock

owned by root

The database transaction authority serializing migration callers for one schema.

Migration watermark

owned by root

The greatest recorded schema migration version used to select forward migrations. It is not a schema fingerprint or a compatibility proof.

Owner token

owned by root

The adapter-issued opaque proof distinguishing one ownership claim from another claim for the same execution. It grants no independent application authentication.

PostgreSQL store

owned by root

The adapter-owned persistence scope identified by one database connection target and one schema. It represents Saga storage through execution rows.

Retention

owned by root

The continued availability of saved execution evidence and identity after execution progress stops or finishes.

Schema name

owned by root

The admitted namespace identifier that separates PostgreSQL stores within one database.

Takeover

owned by root

Replacement of an expired current claim by a new claim with a greater generation and a distinct owner token.